String Operation:
1.strlen()
2.strcpy()
3.strncpy()
4.strcat()
5.strcmp()
6.strcmpi()
7.strncmp()
8.strlwr()
9.strupr()

Example Program:
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main()
{
char *str1,*str2,*str3[100],temp[100];
int len;
printf("Enter the first String: ");
gets(str1);
printf("Enter the second String: ");
scanf("%s",str2);
printf("Length of Str1=%s:",strlen(str1));
printf("%d",length of Str2=%s",strlen(str2));

strcpy(temp,str1);
printf("\nCopy of str1=%s",temp);
strcpy(str1,str2);
strcat(temp,str1);
printf("\n Concatenation %s",temp);
strncpy(str1,str2,2);
printf("\nstr1 are same string");
else
printf("\nstr1 are not same string");

strcpy(temp,str1);
for(i=0;temp[i]!='\0';i++)
temp[i]=toupper(temp[i]);
puts(temp);
printf("\n Lowercase %s",temp);
strcpy(temp,str1);
for(i=0;temp[i]!='\0';i++)
temp[i]=tolower(temp[i]);
puts(temp);
printf("\n Reverse %s",temp);
strcpy(temp,str1);
printf("\n Palindrome");
else
printf("\n Not Palindrome");
return 0;
}

Character Operations:
#include<stdio.h>
1.isalpha(ch)
2.isdigit(ch)
3.isalnum(ch)
4.isspace(ch)
5.islower(ch)
6.isupper(ch)
7.isprint(ch)
8.iscntrl(ch)
9.isxdigit(ch)

Example code:
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
int main()
{
char ch;
printf("Enter a character: ");
scanf("%c",&ch);
printf("\n");
printf("%c is alphabetic",
if(isalpha(ch))
printf("%c is digit",
printf("Lowercase %c",islower(ch));
printf("Uppercase %c",isupper(ch));
return 0;
}

Pointer:
It stores the address of variables.
Types of Pointer:
1.NULL Pointer
A pointer that points nothing.
Example code:
#include<stdio.h>
#include<ctype.h>
int main()
int *p=NULL;
if(p==NULL)
printf("NULL pointer");
}
return 0;
}

2.Void Pointer(Generic Pointer):
A pointer that can hold address of any data types.
Example code:
#include<stdio.h>
#include<ctype.h>
int main()
int a=10;
void *p=&a;
int *ip;
ip=(int *)p;
printf("Value of a %d\n",*(int*)p);
p=&ch;
printf("Value of a %c\n",*(char*)p);
return 0;
}

3.Wild Pointer:
Uninitialized Pointer(contains garbage address)
Example code:
#include<stdio.h>
#include<ctype.h>
int main()
int *p;
printf("%d\n",*p);
printf("Wild Pointer created");
return 0;
}

4.Dangling Pointer:
Pointer pointing to a memory that is freed
Off out of scope.
Example code:
#include<stdio.h>
#include<stdlib.h>
int main()
int *p=(int *)malloc(sizeof(int));
*p=10;
free(p);
printf("%d",*p);
return 0;
}

5.Pointer to Pointer(Double Pointer):
Pointer that stores the address of another pointer.
Example code:
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
int main()
int a=10;
int *p=&a;
int **ip=&p;
printf("%d\n",a);
printf("%d\n",*&a);
printf("%d\n",*p);
printf("%d\n",**ip);
printf("%d",*(*ip));
return 0;
}

6.Function Pointer:
Stores the address of a function.
Example code:
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
int add(int,int);
int main()
int a=10,b=20,sum;
int *p(int,int);
p=&add;
sum=*p(a,b);
printf("%d",sum);
return 0;
}

7.Array Pointer:
Pointer that holds address of an entire array.
Example code:
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
int main()
int arr[5]={10,20,30};
int *p[5]={&arr};
printf("%d",*(*p[0]));
return 0;
}

Call By Value:
#include<stdio.h>
void swap(int a,int b);
int temp=a;
a=b;
b=temp;
}

Call by Reference:
#include<stdio.h>
void swap(int *a,int *b)
{int temp=*a;
*a=*b;
*b=temp;
}
int main()
{int a=10,p=20;
swap(&a,&p);
printf("%d%d",a,p);
return 0;
}

Structure:
#include<stdio.h>
struct Student{
int roll;
char name[20];
float marks;
};
int main()
{
struct Student s1={101,"ABC",85.5};
printf("%d\n",s1.roll);
printf("Name %s\n",s1.name);
printf("Marks %f\n",s1.marks);
printf("%d\n",s1.roll);
printf("Name %s\n",s1.name);
printf("%d\n",s1.roll);
printf("Name %s\n",s1.name);
return 0;
}

UNION:
#include<stdio.h>
union Student{
int roll;
float marks;
int cid;
};
int main()
{
union Student s1;
s1.roll=101;
printf("%d\n",s1.roll);
s1.marks=85.5;
printf("Marks %f\n",s1.marks);
printf("%d\n",s1.roll);
printf("Marks %f\n",s1.marks);
printf("%d\n",s1.roll);
s1.cid=303;
printf("%d\n",s1.cid);

return 0;
}

DYNAMIcmemo Memory Allocation:
1.malloc()
example code:
#include<stdio.h>
#include<stdlib.h>
int main()
int n,i;
int *p;
printf("Enter the number of elements:");
scanf("%d",&n);
p=(int*) malloc(n*sizeof(int));
if(p==NULL)
exit("Not Allocated.\n");
return 1;
}
printf("Enter %d elements:",n);
for(i=0;i<n;i++)
scanf("%d",&p[i]);
}
printf("You entered:\n");
for(i=0;i<n;i++)
printf("%d ",p[i]);
}
free(p);
return 0;
}

2.calloc()
example code:
#.free()
Example code:
#include<stdio.h>
#include<stdlib.h>
int main()
int n,i;
int *p;
printf("Enter the number of elements:");
scanf("%d",&n);
p=(int*) calloc(n,sizeof(int));
if(p==NULL)
exit("Not Allocated.\n");
return 1;
}
printf("Enter %d elements:",n);
for(i=0;i<n;i++)
scanf("%d",&p[i]);
}
printf("You entered:\n");
for(i=0;i<n;i++)
printf("%d ",p[i]);
}
free(p);
return 0;
}

3.realloc()
Example code:
#include<stdio.h>
#include<stdlib.h>
int main()
int n,i,*p,*q;
printf("Enter Initial Value:");
scanf("%d",&n);
p=(int*) malloc(n);
if(p==NULL)
exit("Not Allocated.\n");
return 1;
}
printf("Enter Initial Value:");
scanf("%d",&n);
p=(int*) realloc(p,n*sizeof(int));
printf("Enter first elements:");
for(i=0;i<n;i++)
scanf("%d",&p[i]);
}
printf("Your updated Arrays:\n");
for(i=0;i<n;i++)
printf("%d",p[i]);
}
free(p);
return 0;
}

File Handling:
File Operations:
1.fopen() –opens file in a specific mode.
2.fprintf() – writes formatted data to a file.
3.fscanf() – reads formatted data from a file.
4.fgetc()/fputc() – reads and write a single character.
5.fgets()/fputs() – reads and writes a strings.
6.fclose() – closes the file.

Functions of fopen()mode of the operation:
"r,w" – works in a file
"r+" – reads the file
"r,w" – rapped the file
"a,w+" – reads and read the file
"r+" – reads and writes the file
"a,w+" – rapped and reads the file
Write a file

#include <stdio.h>
int main(){
FILE *fptr;
char name[100];
int age;
fptr = fopen("student.txt", "w"); // Open for writing
if (fptr == NULL)
{ printf("Error opening file\n");
return 1;
}
printf("Enter name: ");
gets(name);
printf("Enter age: ");
scanf("%d", &age);
fprintf(fptr, "Name: %s\nAge: %d\n", name, age);
fclose(fptr);
printf("Data written successfully\n");
return 0;
}

Read a file

#include <stdio.h>
int main(){
FILE *fptr;
fptr = fopen("student.txt", "r");
int age;
fptr = fopen("student.txt", "r"); // open file in read mode
if (fptr == NULL)
{ printf("Error opening file\n");
return 1;
}
fprintf(fptr, "Name: %s\nAge: %d\n", name, age);
fclose(fptr);
printf("Data appended successfully\n");
return 0;
}

Append a file

#include <stdio.h>
int main(){
FILE *fptr;
fptr = fopen("student.txt", "a");

if (fptr == NULL)
{ printf("Error opening file\n");
return 1;
}
fprintf(fptr, "Course: B.Tech\n");
fclose(fptr);
printf("Data appended successfully\n");
return 0;
}